

JavaScript, операторы языка

JavaScript

JavaScript - интерпретатор с элементами объектно-ориентированной модели. JavaScript делает web-странички «живыми».

В браузере JavaScript умеет делать все, что относится к манипуляции со страницей, взаимодействию с пользователем, а именно:

- создавать новые HTML-теги, удалять существующие, менять стили элементов, прятать, показывать элементы и т.д.;
- реагировать на действия пользователя, обрабатывать клики мыши, перемещения курсора, нажатия на клавиатуру и т.д.

Язык JavaScript является регистрозависимым.

Программы, написанные на нем, называются скриптами.

Популярность

JavaScript входит в число самых популярных языков программирования.

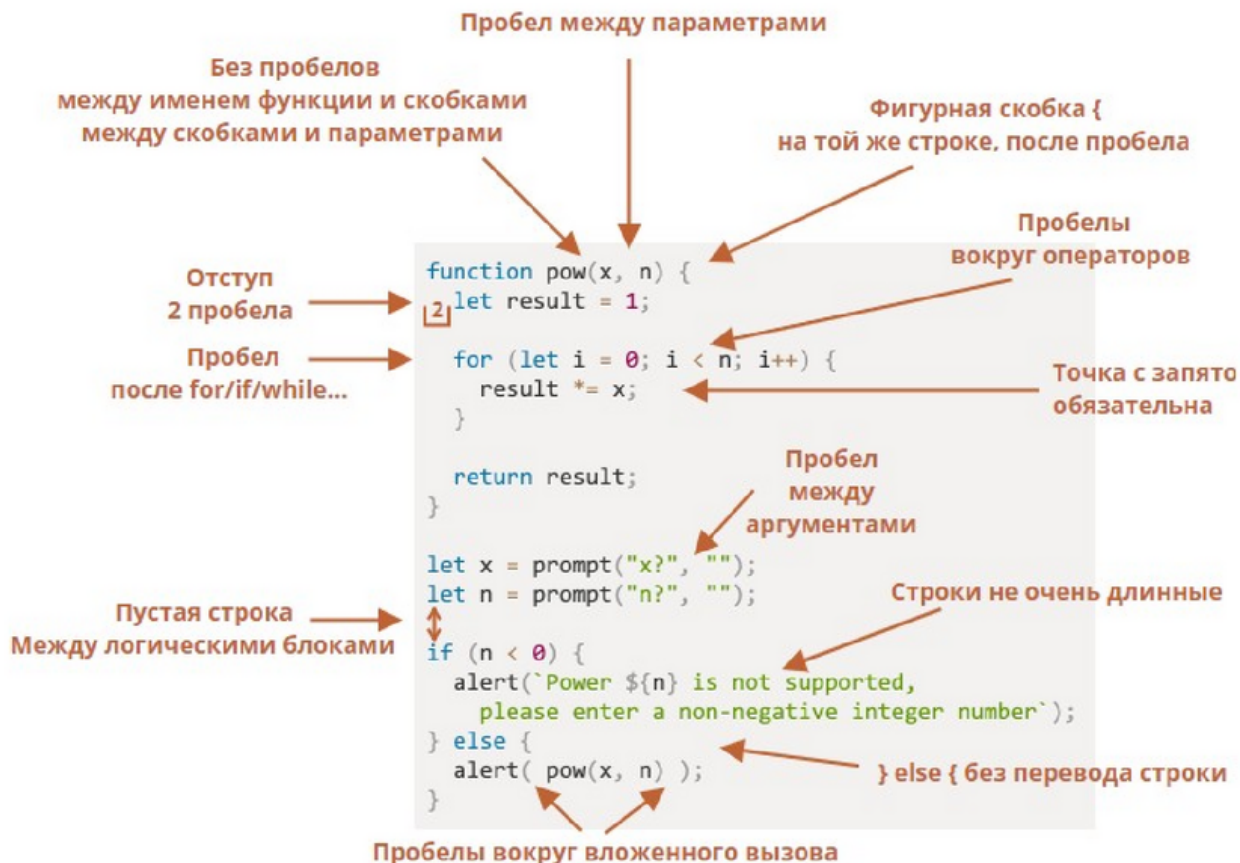
В марте 2023 года JS занимает седьмое место в индексе TIOBE, который составляется на основе статистики поисковых запросов.

Этот язык популярнее, чем PHP, Swift, Ruby, Go.

Популярность

Programming Language	2023	2018	2013	2008	2003	1998	1993	1988
Python	1	4	8	7	13	25	17	-
C	2	2	1	2	2	1	1	1
Java	3	1	2	1	1	18	-	-
C++	4	3	4	4	3	2	2	6
C#	5	5	5	8	10	-	-	-
Visual Basic	6	15	-	-	-	-	-	-
JavaScript	7	7	11	9	8	22	-	-
SQL	8	251	-	-	7	-	-	-
PHP	10	8	6	5	6	-	-	-
Lisp	29	31	13	16	14	7	4	3
Pascal	211	140	15	20	99	12	3	14
(Visual) Basic	-	-	7	3	5	3	8	5

Стиль написания кода



Размещение JavaScript-код

JavaScript – язык, установленный по умолчанию в браузере:

```
<script>  
  код программы  
</script>
```

Также можно создать файл с расширением ***.js**, писать программы в нем и присоединить его к нужной HTML-странице с помощью следующего тега (текст программы сохранен в файле **program.js**):

```
<script type="text/javascript" src="program.js"></script>
```


Комментарии

«Если вам кажется, что нужно добавить комментарий для улучшения понимания, это значит, что ваш код недостаточно прост, и, может, стоит переписать его» (Р. Мартин, «Чистый код»)

Для вставки однострочного комментария в программу используются символы "//":

// однострочный комментарий

Многострочные комментарии располагаются между символами "/*" и "*/":

/* комментарий, начало

...

комментарий, конец */

Вложенные комментарии в JavaScript не поддерживаются.

Вывод информации

Вывод информации в виде всплывающего окна в браузере :

alert(сообщение);

Вывод информации в консоль:

console.log(сообщение);

Вывод информации на HTML-страницу:

document.write(сообщение, допускаются теги);

Задание 1

Задание 1. Написать программу, которая выводит сообщение *'Hello, world!'* на HTML-страницу.

Переменные

Переменные создаются при помощи операторов **let**, если в программе эту переменную нужно изменять, или **const**, если значение переменной изменять не нужно:

let имяПеременной;

или

const имяПеременной;

Соглашения на имена

Существует несколько **обязательных правил** для выбора имен переменных.

- Имя должно состоять из букв, цифр и знаков '_' или '\$'.
- Не может начинаться с цифры.
- Не может совпадать с ключевыми словами языка.
- JavaScript различает большие и маленькие буквы в имени (переменная `x` и `X` – разные переменные).

Также есть рекомендации по выбору имен:

- Имя переменной должно максимально точно соответствовать хранимым в ней данным.
- Имена переменных могут состоять из нескольких слов, тогда они пишутся в "верблюжьей нотации" **воТак** (все слова, кроме первого, начинаются с большой буквы).

Операторы присваивания

Переменной можно присвоить некоторое значение с помощью оператора присваивания:

```
let имяПеременной;  
имяПеременной = выражение;
```

Или переменной можно присвоить значение при ее создании:

```
let имяПеременной = выражение;
```

Не допускается повторное использование оператора `let` (`const`).

Тип переменной

Тип данных переменной определяется в зависимости от выражения (или значения), которое ей было присвоено, либо при создании переменной, либо при использовании оператора присваивания.

При этом тип переменной в процессе выполнения программы может изменяться.

Число

Тип данных **number** может использоваться для описания переменных целого и вещественного типа:

```
let n;  
n = 10;  
let error = 0.0003;  
error = 0.3e-3; //0.3*10-3
```

Все числа в JavaScript (и целые, и вещественные) хранятся в 64-битном формате IEEE-754, который также называют «числа с плавающей точкой двойной точности».

Строка

Строкой считается последовательность символов, заключенных в одинарные ('), двойные кавычки (") или обратные кавычки (`):

```
let webTools = 'HTML CSS JavaScript';  
let webTools = "HTML CSS JavaScript";  
let webTools = `HTML CSS JavaScript`;
```

Двойные и одинарные кавычки работают одинаково, содержимое в них может располагаться только на одной строке.

Обратные (``) кавычки могут занимать более одной строки.

Логический тип

Переменная данного типа может принимать два значения: **true** (истина) и **false** (ложь).

```
let sorted = true;  
  
sorted = false;
```

Выражения

Выражения языка JavaScript могут включать:

- переменные;
- константы;
- вызовы функций;
- бинарные операции;
- круглые скобки.

Бинарные операции

Арифметические операции:

- + - сложение;
- - вычитание;
- * - умножение;
- / - деление;
- % - остаток от деления (применяется только к целым числовым значениям);
- ** - возведение в степень.

Бинарные операции

Операции отношения:

`==` - равно;

`===` - равно не только значение, но и тип

`!=` - не равно;

`!==` - не равно (либо значение, либо тип);

`>=` - больше или равно;

`<=` - меньше или равно;

`>` - больше;

`<` - меньше.

Бинарные операции

Логические операции:

`&&` - и,

`||` - или

`!` – логическое отрицание

Приоритеты операций

Приоритеты операций:

1. круглые скобки ();
2. возведение в степень (**)
3. умножение и деление (*, /, %);
4. сложение и вычитание (+, -);
5. операции сравнения (>, >=, <, <=);
6. равно и не равно (==, !=);
7. логическое отрицание (!);
8. логическое **И** (&&);
9. логическое **ИЛИ** (| |).

Вычисление выражений

Какое значение получит переменная:

```
let a = 7;
```

```
let b = 4;
```

```
let result;
```

```
result = a * (b - 2) ** 2; // результат: 28
```

```
result = a % 3 + b % 2; // результат: 1
```

```
result = a % b ** 2; // результат: 7
```

```
result = (a % b) ** 2; // результат: 9
```

```
result = (b - a) / b * 3; // результат: -2.25
```


Вычисление выражений

Какое значение получит переменная:

```
let a = 12;
```

```
let b = 5;
```

```
let result;
```

```
result = a == b * 2; // результат: false
```

```
result = a % 3 == b % 5; // результат: true
```

```
result = !(a % b ** 2 > b * 2); // результат: false
```

```
result = (a == b || b > 0) && !(a > b); // результат: false
```

```
result = a == b || b > 0 && (a > b); // результат: true
```

Вещественные числа

При операциях с вещественными числами накапливается ошибка (и это не только в JavaScript).

$2 + 0.3 + 0.3 + 0.3$ – ожидаемый результат 2.9

JavaScript:

```
alert(2 + 0.3 + 0.3 + 0.3)
```

Результат:

2.8999999999999995

Python:

```
print(2 + 0.3 + 0.3 + 0.3)
```

Результат:

2.8999999999999995

Вещественные числа

Числа 2.3 и 2.8999999999999995 с точностью 0.0000000000000001 равны между собой.

Вещественные числа сравнивать, особенно на равенство НЕЛЬЗЯ.

Их можно сравнивать только с некоторой погрешностью:

$$a == b \Rightarrow |a - b| < 0.000001$$

Вещественные числа

Рекомендации

1. Если известно, что вещественное число должно содержать точное количество знаков после запятой (например, как в случае с деньгами), то число в программе хранится и используется как целое ($a * 100$), а при выводе – делится на 100.

Пример:

$$2 + 0.3 + 0.3 + 0.3 \Rightarrow 200 + 30 + 30 + 30 = 290$$

$$290 / 100 \Rightarrow 2.9$$

Операции со строками

Соединение строк

Соединение строк осуществляется с помощью оператора "+":

```
let str1 = 'Script';
let str2 = 'Java';
alert(str2 + str1); // выведет JavaScript
```

Операции со строками

Для вставки в строку значений выражений и переменных можно использовать:

Способ 1. Соединение строк

```
let version = 3;
let webTools = "HTML" + (version+2)+ " CSS" + version + " JavaScript";
alert(webTools); //результат - HTML5 CSS3 JavaScript
```

Способ 2

В строки с обратными кавычками (``) можно включать переменные и выражения, заключенные в \${...}:

```
let version = 3;
let webTools = `HTML${ version + 2 } CSS${ version } JavaScript`;
alert(webTools); //результат HTML5 - CSS3 JavaScript
```

Операции со строками

Задание 2. Написать код, который выводит сообщение пользователю на страницу в виде заголовка уровня 3 вида:

Доброго времени суток, *имя_пользователя*! Мы отправили Вам приглашение на почту *email_пользователя*.

Имя пользователя и его email занесены в переменные.

```
let name = "Алина";  
let email = "alina@test.ru";
```

Преобразование к числу

JavaScript поддерживает явное и неявное преобразование типов.

Неявное преобразование **из строки в число** происходит в математических функциях и выражениях:

```
let result = '21' - '5'; // 16  
let result = '21' / '5'; // 4.2
```

Но с неявным преобразованием нужно быть осторожным, например:

```
let result = '21' + '5'; // 215
```

Преобразование к числу

Для преобразования к числу в явном виде можно использовать

- символ +;
- функцию **Number()**

```
let x = +'21'; // 21
let x = Number('21'); // 21
```

Преобразование из строки в число происходит следующим образом:

- обрезаются пробельные символы сначала и конца строки;
- если остается пустая строка, то результатом будет 0;
- если строка, которая описывает число, то результат – это число;
- в противном случае – **NaN**.

Преобразование к числу

Для преобразования к числу в явном виде можно использовать

- **parseFloat()**
- **parseInt()**

```
result = parseFloat('12.9'); // 12.9
result = parseInt('12.9'); // 12
result = parseFloat(''); // NaN
```

Во всех случаях, если введено не число, результат преобразования - NaN.

Особенность этих функции в том, что они пытаются преобразовать в число переданное значение до тех пор, пока не встретят символ, который не интерпретируется в число.

При этом, если первые символы - число, то это число и возвращается.

```
result = parseFloat('12.9x_z'); // 12.9
```

Преобразование к числу

Чему будут равно значения переменной:

```
let result;

result = +'12'; // 12
result = Number(''); // 0
result = +'12abc'; // NaN
result = Number(null); // 0

result = parseFloat('12.9'); // 12.9
result = parseInt('12.9'); // 12
result = parseInt('12abc'); // 12
result = parseInt(''); // NaN
result = parseFloat('abc'); // NaN
result = parseFloat(null); // NaN
```

Преобразование к строке

Происходит, когда требуется представление чего-либо в виде строки.

Это преобразование можно выполнить **явным образом**, используя функцию **String()**, неявным - его производит оператор **alert()** или **document.write()**.

```
let f = true;
alert(f); // 'true'
alert(23 * 6); // '138'
let a = String(3); // '3'
```

Преобразование происходит способом «как есть»: **false** становится **'false'**, **true** - **'true'**, число - **'число'**.

Специальные функции

Функция `isNaN()`

С помощью функции `isNaN()` проверяет, равно ли **NaN** переданное ей значение.

Если равно - функция возвращает **true**, в противном случае - **false**.

При этом нечисловое значение функция пытается преобразовать к числу.

Если это возможно, то результат **false**, если невозможно - **true**.

Специальные функции

Чему будет равно значение переменной:

```
let result;

result = isNaN('12.9'); // false
result = isNaN(12); // false
result = isNaN(false); // false
result = isNaN('abs'); // true
result = isNaN(''); // false
result = isNaN(null); // false
result = isNaN(NaN); // true
```

Специальные функции

Функция `isFinite()`

Функция **`isFinite()`** определяет, является ли переданное параметром значение конечным числом. При этом она пытается преобразовать переданное значение в число.

Эта функция возвращает **`false`**, если значение переданного параметра равно **`+infinity`**, **`-infinity`**, или **`NaN`**, в противном случае она возвращает **`true`**.

Специальные функции

Чему будет равно значение переменной:

```
let result;  
  
result = isFinite('12.9'); // true  
  
result = isFinite(12.9); // true  
  
result = isFinite('abs'); // false  
  
result = isFinite(''); // true, пустая строка преобразуется к 0
```

Ввод данных `prompt()`

Функция **`prompt()`** выводит модальное окно с заголовком, полем для ввода текста, и кнопками «ОК» и «Отмена».

Для ввода данных в переменную используются операторы:

`let` имяПеременной = `prompt(подсказка, значение_по_умолчанию);`

Результат ввода – строка, которая заносится в переменную.

Пример. Спросим у пользователя его имя.

```
const name = prompt('Введите имя:', 'Алина');
```

Проверка, является ли строка числ

При вводе числового значения с помощью функции **`prompt()`**, может произойти следующее:

- введено число;
- введено не число (ошибочное число);
- ничего не введено;
- нажата кнопка отмена.

Проверка, является ли строка числом

Выражение:

```
isNaN(parseFloat(переменная))
```

имеет значение **true**, если введено ошибочное число, ничего не введено или нажата кнопка **Отмена**.

Но эта проверка не работает, если строковая переменная начинается с цифр, например `isNaN(parseFloat('12abc'))` возвращает **false**, так как `parseFloat('12abc') = 12`.

Проверка, является ли строка числом

Для проверки, введено ли пользователем верное число с помощью функции `prompt()`, рекомендуется использовать выражение:

```
isNaN(parseFloat(переменная)) || !isFinite(переменная)
```

Это выражение возвращает **false**, если введено число, и **true** во всех остальных случаях.

Базовые операторы языка JavaScript

Операторы присваивания

Переменной можно присвоить некоторое значение с помощью оператора присваивания:

```
let имяПеременной;  
имяПеременной = выражение;
```

Или переменной можно присвоить значение при ее создании:

```
let имяПеременной = выражение;
```

Не допускается повторное использование оператора **let** (**const**).

Унарные операторы

Инкремент и декремент:

- **++** увеличивает значение переменной на 1;
- **--** уменьшает значение переменной на 1.

Унарные операторы

Совмещённые операторы присваивания:

- **+=** - складывает значения левого и правого операндов и присваивает результат левому операнду;
- **-=** - вычитает значения левого и правого операндов и присваивает результат левому операнду;
- ***=** - умножает значения левого и правого операндов и присваивает результат левому операнду;
- **/=** - делит значения левого операнда на правый и присваивает результат левому операнду;
- **%=** - вычисляет остаток от деления левого операнда на правый и присваивает результат левому операнду.

Операторы присваивания

Вычислите значения выражений

```
let z = x = y = 10;
```

```
z--; // 9
```

```
z += z % 6; // 12
```

```
x %= (y - 2); // 2
```

```
y -= x * 2 % 3; //9
```

Условный оператор

Условный оператор предназначен для реализации возможности выполнения различных действий в зависимости от условия.

```
if (условие) оператор;
```

либо

```
if (условие) {  
    оператор;  
}
```



Условный оператор

Условный оператор предназначен для реализации возможности выполнения различных действий в зависимости от условия.

if (условие) оператор;

Этот вариант предпочтительнее

```
if (условие) {  
    оператор;  
}
```



Условный оператор

Полная запись условного оператора:



```
if (условие) {  
    оператор_1;  
    оператор_2;  
} else {  
    оператор_3;  
    оператор_4;  
}
```

Условный оператор

Запись условного оператора с **else if**:

```
if (условие) {  
    оператор_1;  
    оператор_2;  
} else if (условие_1) {  
    оператор_3;  
    оператор_4;  
} else if (условие_2) {  
    оператор_5;  
    оператор_6;  
} else {  
    оператор_7;  
    оператор_8;  
}
```

Условный оператор

Задание 3. Ввести значение переменной **n**, проверить, является ли оно числом. Если введено не число – вывести ошибку, в противном случае вывести увеличенное на 1 значение переменной **n**.

Тернарный оператор

переменная = (условие) ? выражение1 : выражение2;

Выполняется этот оператор следующим образом:

- проверяется условие, если условие истинно – вычисляется выражение1 и полученное значение присваивается переменной,
- в противном случае переменной присваивается значение выражения2.

Пример

```
let typeTime = (hours >= 12) ? "PM" : "AM";
```

Тернарный оператор

Задание 4. Записать следующий условный оператор в виде тернарного оператора.

```
let result;

if (n > 0) {
  result = 1 / n;
} else if (n < 0) {
  result = 1 / n ** 2;
} else {
  result = 0;
}
```

Цикл for

```
for ([инициализация нач. знач.;;] [условие;;] [шаг]) {  
    ...  
}
```

Все параметры оператора **for** являются необязательными и используются для управления процессом выполнения цикла. При применении параметров каждую часть нужно отделять точкой с запятой (;).

Пример

```
for (let i = 1; i <= 3; i++) {  
    document.write(`i = ${i} `);  
}
```

Результат:

```
i = 1  
i = 2  
i = 3
```

Цикл while

```
while (условие) {  
    ...  
}
```

Если при входе в цикл условие ложно, тело цикла не выполнится ни разу.

Пример

```
let i = 0;  
while (i <= 3) {  
    alert(`i = ${i}`);  
    i++;  
}
```

Результат:

```
i = 0  
i = 1  
i = 2  
i = 3
```

Цикл do

```
do {  
    ...  
} while (условие);
```

Цикл выполняется до тех пор, пока условие - истина. Проверка условия повторения осуществляется после выполнения операторов в теле цикла.

Тело цикла **do** всегда выполняется хотя бы один раз.

Результат:

Пример

`i = 0`

```
let i = 0;  
do {  
    alert(`i = ${i}`);  
    i++;  
} while (i < 0);
```

Пример

Пример. Построить сетку значений на интервале **[a, b]**, количество узлов – **n**.

```
const a = -5;  
const b = 5;  
const n = 15;  
const h = (b - a) / (n - 1);
```

Пример

```
/* вариант 1 */
for (let i = 0; i < n; i++) {
    let x = a + i * h;
    document.write(`${ x }<br>`);
}

/* вариант 2 */
let x = a;
while (x <= b) {
    document.write(`${ x }<br>`);
    x +=h;
}
```

Пример

$h = 0.7142857142857143$

Вариант 1

-5
-4.285714285714286
-3.571428571428571
-2.857142857142857
-2.142857142857143
-1.428571428571429
-0.714285714285714
0
0.714285714285714
1.428571428571429
2.142857142857143
2.857142857142857
3.571428571428571
4.285714285714286
5

Вариант 2

-5
-4.285714285714286
-3.571428571428571
-2.857142857142857
-2.142857142857143
-1.428571428571429
-0.714285714285714
6.661338147750939e-16
0.714285714285714
1.428571428571429
2.142857142857143
2.857142857142857
3.571428571428571
4.285714285714286
5.000000000000001

Досрочный выход из цикла

Для досрочного выхода из цикла используется оператор **break**.

Выполнение цикла прекращается в той точке, в которой размещен этот оператор.

Управление передается следующему оператору, находящемуся непосредственно после цикла.

Переход к следующей операции

С помощью оператора **continue** можно прекратить выполнение текущей итерации цикла и перейти к следующей.

```
let sum = 0;
let html = "Нечетные числа:<br>";

for (let i = 0; i < 30; i++) {
  if (i % 2 === 0) continue;
  html += `${i} `;
  sum += i;
}
document.write(`${html}<br>Их сумма: ${sum}`);
```

Переход к следующей операции

Эквивалентный код без **continue**:

```
let sum = 0;
let html = "Нечетные числа:<br>";

for (let i = 0; i < 30; i++) {
  if (i % 2 === 0) {
  } else {
    html += `${i} `;
    sum += i;
  }
}
document.write(`${html}<br>Их сумма: ${sum}`);
```

Циклы

Задание 5. Написать программу, которая формирует на странице поле со списком SELECT. В качестве значений **option** вывести нечетные квадраты чисел, не превышающие **n**.

Например, для **n=81**, поле со списком должно выглядеть так:

```
<select>
  <option>1</option>
  <option>9</option>
  <option>25</option>
  <option>49</option>
  <option>81</option>
</select>
```

Функции

Функции решают три важные задачи:

- избавляют от необходимости многократно повторять в тексте программы аналогичные фрагменты;
- улучшают структуру программы, облегчая ее понимание;
- повышают устойчивость к ошибкам программирования.

Имена функций

Имя функции следует выбирать по тем же правилам, что и имя переменной. Основное отличие – в ее названии должен присутствовать глагол, так как функция – это действие.

Как правило, используются глагольные префиксы, обозначающие общий характер действия, после которых следует уточнение.

```
showMessage( . . ) // префикс show, "показать" сообщение  
getName( . . ) // префикс get, "получает" имя  
calcNOD( . . ) // calc, "вычисляет" наибольший общий делитель  
setValue( . . ) // set, "задает" значение  
checkRead( . . ) // check, "проверяет" ввод, возвращает true/false
```

Функции

Синтаксис описания функции

```
function имя([список формальных параметров]) {  
    оператор_1;  
    оператор_2; ...  
    return ...  
}
```

Синтаксис вызова:

```
имя([список фактических параметров]);
```

Формальные параметры

Параметрам функции можно присваивать значения по умолчанию:

```
function имя(параметр1, параметр2=значение2) {  
    ...  
}
```

Пример

```
function getSum(a, b=1) {  
    return a + b  
}
```

```
alert( getSum(2, 3) ); // результат 5
```

```
alert( getSum(2) ); // результат 3
```

Формальные параметры

При описании функции можно указывать не все формальные параметры или вообще их не указывать.

Тогда обращение к фактическим параметрам, которые получает функция при вызове, осуществляется через объект **arguments**:

- **arguments[0]** - обращение к первому параметру;
- **arguments[1]** - обращение ко второму параметру;
- и т.д.

Формальные параметры

Пример. Соединим строки, которые будут переданы в качестве параметров.

```
function createString(){
    let html = arguments[0];
    for(let i = 1; i < arguments.length; i++) {
        html += ' ' + arguments[i];
    }
    return html + '.';
}
alert( createString('Я', 'изучаю', 'JavaScript') );
```

Результат:

Я изучаю JavaScript.

Функции

Задание 6. Найти сумму всех нечетных чисел, переданных в качестве параметров.

Function Expression

Function Expression – функциональные выражения

```
function getSum(a, b=1) {  
    return a + b  
}  
alert( getSum(1) ); //результат 2
```



```
let sum = function(a, b=1) {  
    return a + b  
}  
alert( sum(1) ); //результат 2
```

Function Expression

Function Expression не имеют массива **arguments**. Если функция получает неизвестное число параметров, то для их передачи можно использовать массив, перед которым стоят три точки. Имя массива (в примере **args**) может быть любым

```
let sum = function(...args) {  
    return args[0] + args[1];  
}  
alert( sum(1, 2) ); //результат 3
```

Function Expression

Задание 7. Найти сумму всех нечетных чисел, переданных в качестве параметров. Использовать **Function Expression**.

Стрелочные функции

```
let func = (arg1, arg2, ...argN) => expression;
```

Это создаёт функцию **func**, которая принимает аргументы **arg1..argN**, затем вычисляет **expression** в правой части с их использованием и возвращает результат.

Пример:

```
let add = (a, b=1) => a + b  
alert( add(1) ); //результат 2
```

Стрелочные функции

```
let func = () => expression;
```

Если у функции отсутствуют параметры, то при описании и вызове нужно использовать пустые круглые скобки.

Пример:

```
let message = () => alert('Привет!');  
alert(message(1) ); //результат Привет!
```

Стрелочные функции

```
let func = param => expression;
```

Если у функции у функции один параметр, круглые скобки можно не писать.

Пример:

```
let sqrt = x => Math.sqrt(x);  
alert( sqrt(5) ); //результат 2.23606797749979
```

Стрелочные функции

Стрелочные функции могут включать несколько строк, тогда тело функции заключается в фигурные скобки и явно указывается **return**.

Пример:

```
let add = (a, b=1) => {  
    let result = a + b;  
    return result;  
}  
alert( add(1) ); //результат 2
```

Стрелочные функции

```
let func = (...args) => expression;
```

Массива **arguments** у стрелочных функций нет. Если функция получает неизвестное число параметров, то для их передачи можно использовать массив, перед которым стоят три точки. Имя массива (в примере **args**) может быть любым.

Пример:

```
let sumNumber = (...args)=> {  
    let sum = 0;  
    for(let i in args) {  
        sum += args[i];  
    }  
    return sum;  
}  
alert( sumNumber(1, 2, 3, 4, 5) ); //результат 15
```

Стрелочные функции

Задание 8. Найти сумму всех нечетных чисел, переданных в качестве параметров. Использовать стрелочную функцию.

Виды функции

Function Declaration:

```
function getSum(a, b=1) {  
    return a + b  
}  
alert( getSum(1) ); //результат 2
```

Function Expression :

```
let sum = function(a, b=1) {  
    return a + b  
}  
alert( sum(1) ); //результат 2
```

Стрелочная функция:

```
let add = (a, b=1) => a + b  
alert( add(1) ); //результат 2
```

Функции, сравнение

- **Function Declaration** (обычные функции) нельзя занести в другую переменную.
- **Function Expression** (функциональные выражения) и стрелочные функций можно занести в другую переменную.

```
//функциональное выражение  
let sum = function(a, b=1) {  
    return a + b  
}  
//стрелочная функция  
let sub = (a, b=1) => a - b
```

```
add = sum;  
diff = sub;
```

```
console.log( add(1, 2) ); //результат 3  
console.log( diff(1, 2) ); //результат -1
```

Функции, сравнен

- Обычные функции использовать в выражениях нельзя.
- Стрелочные функции и функциональные выражения можно использовать в выражениях.

```
a=3;
b=4;
// Function Expression
let maxNumber = (a > b)?
  function () { return `${ a } > ${ b}` } :
  function () { return `${ a } <= ${ b}` };

alert( maxNumber() ); // 3 <= 4
```

```
// Стрелочная функция
let maxNumber1 = (a > b)?
  () => `${ a } > ${ b}` :
  () => `${ a } <= ${ b}`;

alert( maxNumber1() ); // 3 <= 4
```

Функции, сравнение

- Обычные функции обрабатываются перед выполнением блока кода. Они видны во всём блоке.
- Функциональные выражения и стрелочные функции создаются только тогда, когда поток выполнения достигает их.

Function Declaration:

```
alert( getSum(1) ); //результат 2
function getSum(a, b=1) {
  return a + b
}
```

Стрелочные функция:

```
alert( add(1) ); // Cannot access 'add' before initialization
let add = (a, b=1) => a + b;
```

Функции, сравнение

Независимо от того, как создаётся функция – она является значением.

```
function getSum(a, b=1) {
  return a + b
}
console.log( getSum ); /* f getSum(a, b=1) {
                        return a + b
                        }*/
```

Функции, сравнение

Независимо от того, как создаётся функция – она является значением.

Function Declaration	Function Expression	Стрелочная функция
<pre>function getSum(a, b=1) { return a + b; } alert(getSum); /* результат: function getSum(a, b=1) { return a + b; } */</pre>	<pre>let sum = function(a, b=1) { return a + b; } alert(sum); /* результат: function (a, b=1) { return a + b; } */</pre>	<pre>let sum = (a, b=1) => a + b; alert(sum); /* результат: (a, b=1) => a + b */</pre>

Функции-«колбэки»

Любую функцию можно передать в качестве параметра в другую функцию. При этом при описании формальных и фактических параметров скобки НЕ СТАВЯТЬСЯ.

Пример. Выведем сообщения пользователю после нажатия им кнопки **ОК** или **Cancel** в диалоговом окне.

```
let messageOk = () => alert('Данные сохранены!');
let messageCancel = () => alert('Данные не сохранены!');

function getUserData(isCancel, ok, cancel) {
    return isCancel ? cancel() : ok();
}

alert(getUserData(true, messageOk, messageCancel)); // "Данные не
сохранены!"
alert(getUserData(false, messageOk, messageCancel)); // "Данные
сохранены!"
```

Спасибо за внимание!